PORTFOLIO CASE STUDY

Deckhunter

Designing and shipping a production AI-native mobile app — solo.

iOS & Android

Live on App Store & Google Play

One Piece TCG · Gundam TCG · Riftbound TCG

Cursor & Claude Code

Lavine Dukhan · lavinedukhan.com · 2026

01 THE PROBLEM

Mind-numbingly tedious

My card collection grew and is constantly growing with now over 8000+ cards it meant every time I wanted to build a deck to play I had to find a list comb through all my cards to see what I had and what was missing then keep track of what I needed to get for multiple deck lists. Hours of time over and over again.

The app already knows my collection. Why cant I just give it a decklist and let it tell me what Im missing?

30+ min → ~30 sec

BEFORE

- ✗ Find decklist online
- ✗ Open collection app
- ✗ Switch back to deck list website or picture
- ✗ Check each card manually
- ✗ Write down what's missing
- ✗ Hope you didn't miss anything

WITH DECKHUNTER

- → Import any decklist
- → Instant owned vs. missing analysis
- → One-tap shareable want list image
- → **Done.**

Architect.
Not just a prompt-sender.

I brought my design methodology to the collaboration. The user journeys came first. The PRD was the blueprint. Claude Code was the contractor. I was the architect.

01 DESIGN FIRST

User journeys, information architecture, flows and wireframes before a single line of code.

02 PRD AS BLUEPRINT

Used AI to generate a formal PRD from design docs and ‘interviews’. The PRD governed the build.

03 FEATURE BY FEATURE

One feature at a time. Brief → plan → build → test on real device → iterate → next phase.

04 VERIFICATION AT EVERY STEP

Before each feature was built, I described it and asked: "Is there a better approach?" Final call was always mine.

One app.
Three games.
Each with different rules.

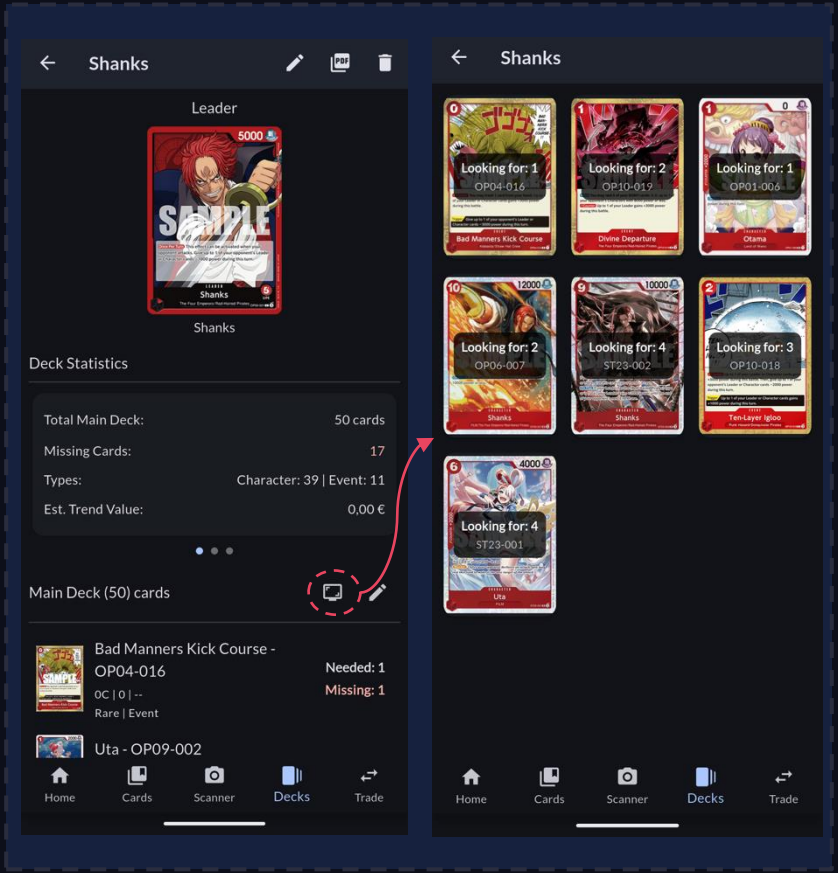
One Piece TCG, Gundam TCG, and Riftbound TCG each have different card schemas, different mechanics, and different deck-building rules. A player who only plays one game shouldn't think about the others at all, players who play all should navigate with ease

The rule: surface the right information at the right moment. Not all card data is equal in all contexts. Inspired by the creation of the cards themselves I refined each process to work across all games providing the user with consistent app behavior

Designed for the real world.

> *TCG trading doesn't happen in apps. It happens in trusted social groups like those on Discord servers and WhatsApp groups. The design question wasn't 'what looks good in the app?' — it was **'what format integrates into how people actually trade cards?'***

One tap. Generates a clean image of every missing card with quantities, ready to drop into any chat. No manual listing. No hunting for card images. Always Accurate.



04 PROTOTYPING METHOD

3 live branches. Not 3 Figma files.

TRADITIONAL APPROACH

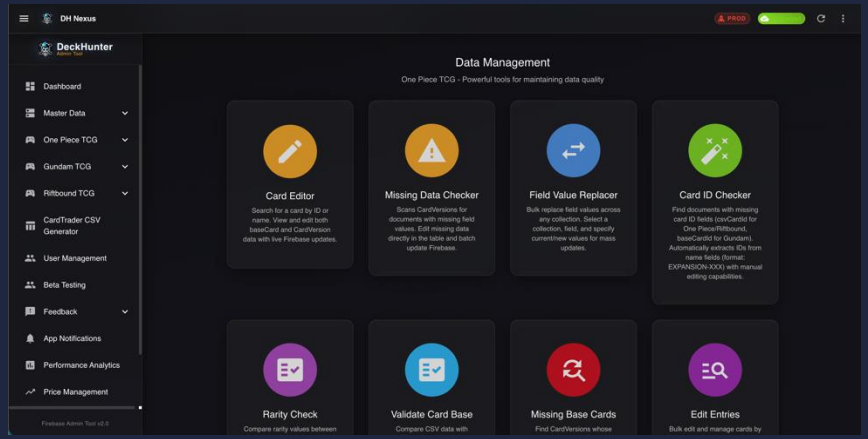
Build assets in Figma
Create interactive prototype
Share link for feedback
Users test simulated interactions
Weeks to test 3 variants

MY APPROACH

Spin up 3 working code branches
Each on a real device simultaneously
Real players test real interactions
Bad ideas killed same day
3 variants tested in hours

The app is half the system.

Three games. Three different release schedules. An entire data pipeline to keep the app fed with accurate, priced card data. I built all of it...well I used Claude to build all of it



DHNEXUS

Admin console

→ Data tools with simulation preview before commit
→ Feedback dashboard — bugs, suggestions, comments
→ Collection health checks across all game data
→ Set management and production publishing
→ *See the result before you commit it*

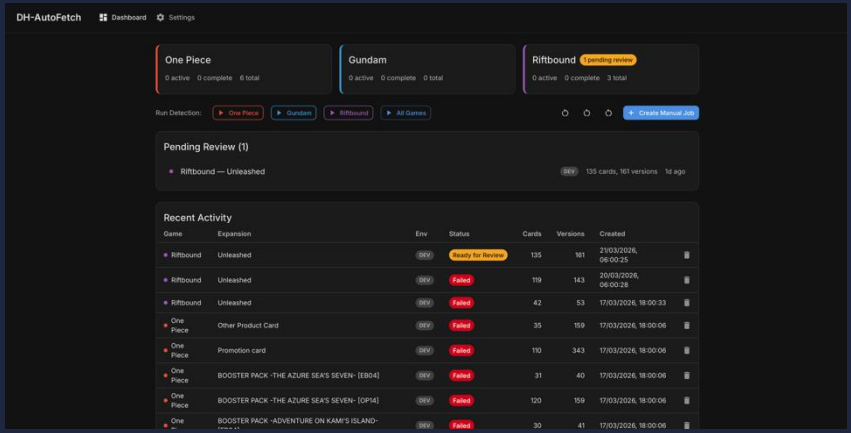
The app is half the system.

Three games. Three different release schedules. An entire data pipeline to keep the app fed with accurate, priced card data. I built all of it...well I used Claude to build all of it

DH-AUTOFETCHER

Automated card data pipeline

→ Monitors official sources for new set releases
→ Fetches and processes data automatically
→ Pulls pricing from external sources
→ Discord webhook notification when ready
→ I review before anything goes to production



When it all comes together

06 “MY OH WOW!” MOMENTS

Things that made me smile.

USERS FOUND IT WITHOUT MARKETING

Organic discovery. Organic paying subscribers. Before a single post went out.

THE FIRST BUG REPORT

A user took the time to report a small bug. The feedback loop I'd designed was working. That felt significant.

"THE TEAM DID A GREAT WORK"

Multiple users wrote in addressing 'the team.' Nobody knew it was one person. That's a signal about the quality bar — not a cute anecdote.

AI amplifies judgment. It doesn't replace it.

WHAT I BROUGHT

- → Design thinking: user journeys, IA, interaction decisions
- → Product judgment: what to build, defer, and cut
- → The quality bar: real device testing with real users
- → Operational design: the system behind the screen
- → **Every final decision**

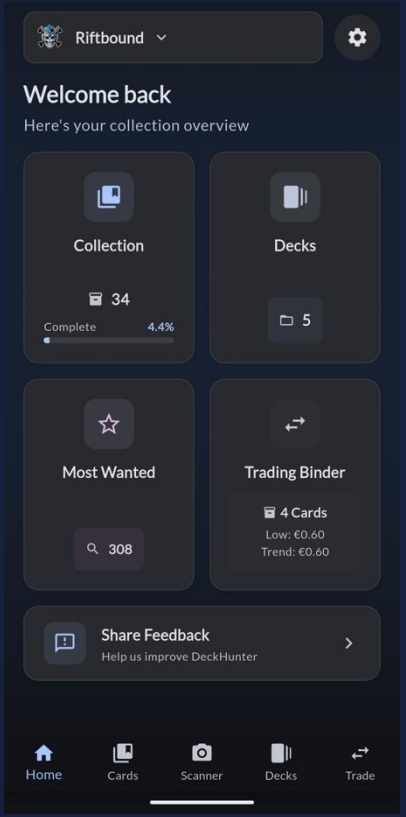
WHAT AI CONTRIBUTED

- → Implementation speed: turning specs into working code
- → Technical research: tradeoffs I couldn't evaluate alone
- → Alternative approaches: pushing back when better options existed
- → Breadth: backend, pipeline, admin console, and app in parallel

DECKHUNTER

Live. Shipping. Users assuming a team built it.

One designer. One AI collaborator. A production mobile app, a data pipeline, and an admin console.



linkedin.com/in/lavine-dukhan-ux